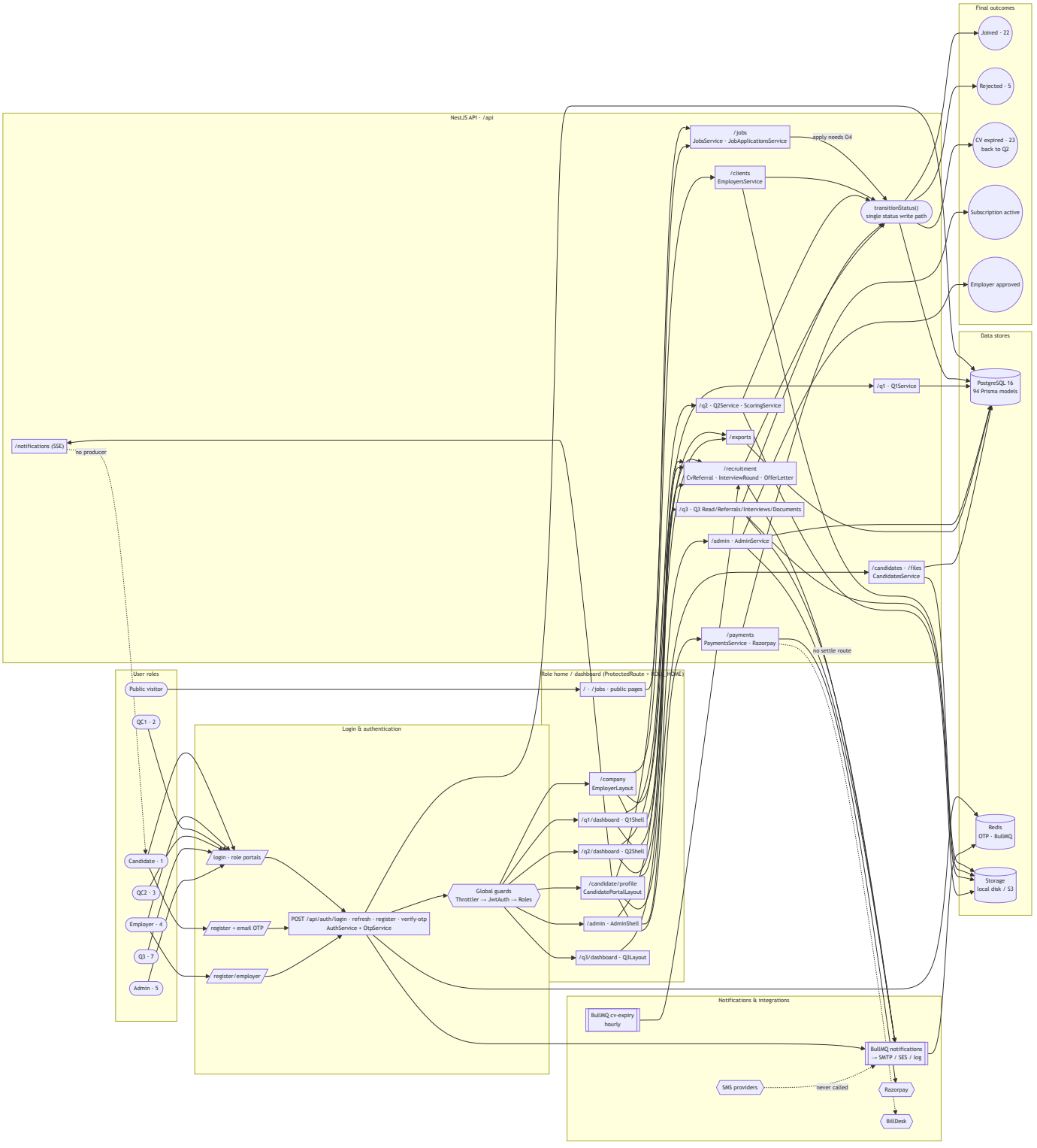
Master System Flow

> One diagram that connects roles, login, dashboards, modules, APIs, services, data stores, notifications and outcomes, followed by focused diagrams for the complex parts. Everything shown exists in code at commit `8de48cf`. Dashed lines mark paths that exist but are broken or unused (see TESTING_AND_ISSUES).

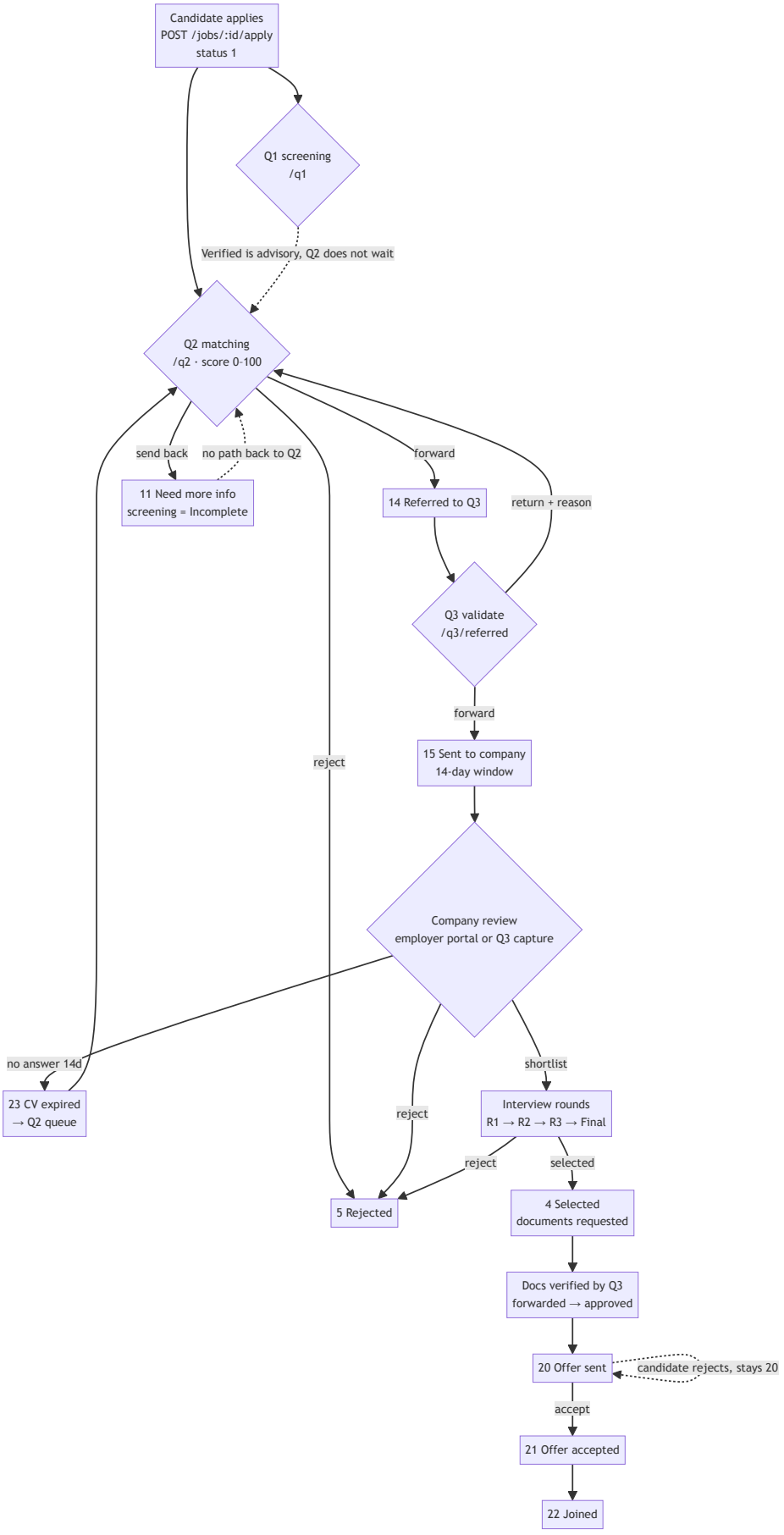
1. Master flow



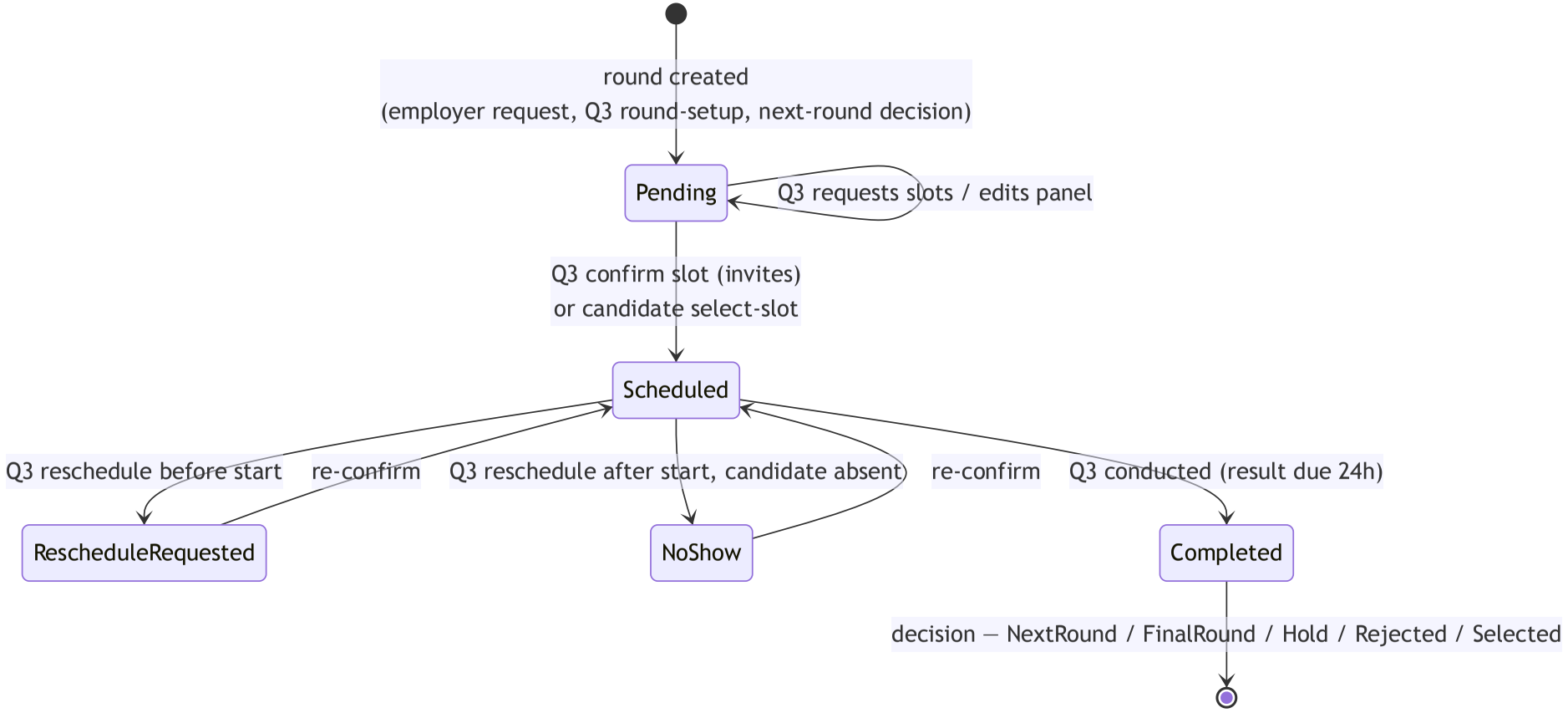
How to read it

1. **Roles → login.** Candidates self-register with email OTP. Employers submit a registration that an admin must approve. Staff accounts are created by an admin.
2. **Guards.** Every API call passes the throttler, the JWT check and the role check (`apps/api/src/app.module.ts`). On the web, `ProtectedRoute` sends each role to its home (`apps/web/src/types/roles.ts`).
3. **Dashboards → modules.** Each workspace calls its own API module. Shared hiring actions live in `/recruitment` .
4. **Status writes.** All application status changes funnel through `JobApplicationsService.transitionStatus` (`apps/api/src/modules/jobs/job-application.service.ts:167`). It writes the mapping, a status log row and the latest-status table in one transaction.
5. **Async.** Emails go through the BullMQ `notifications` queue. The hourly `cv-expiry` job returns unreviewed CVs to Q2 after 14 days.
6. **Outcomes.** These are joined, rejected, expired (re-queued), subscription active and employer approved.

2. Hiring pipeline: hand-offs between desks

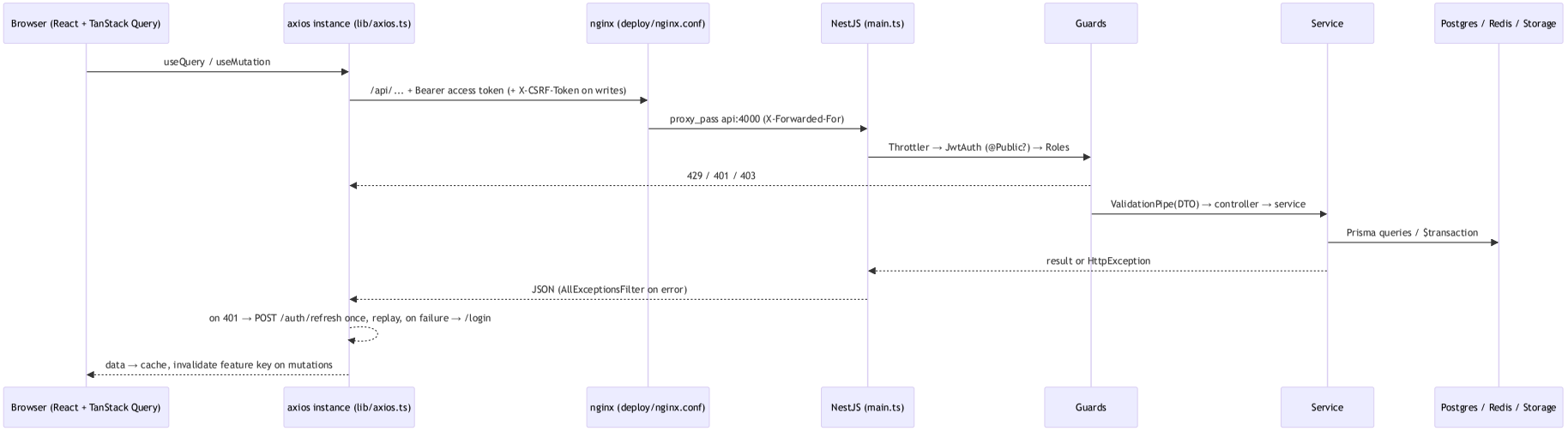


3. Interview round lifecycle (one round)

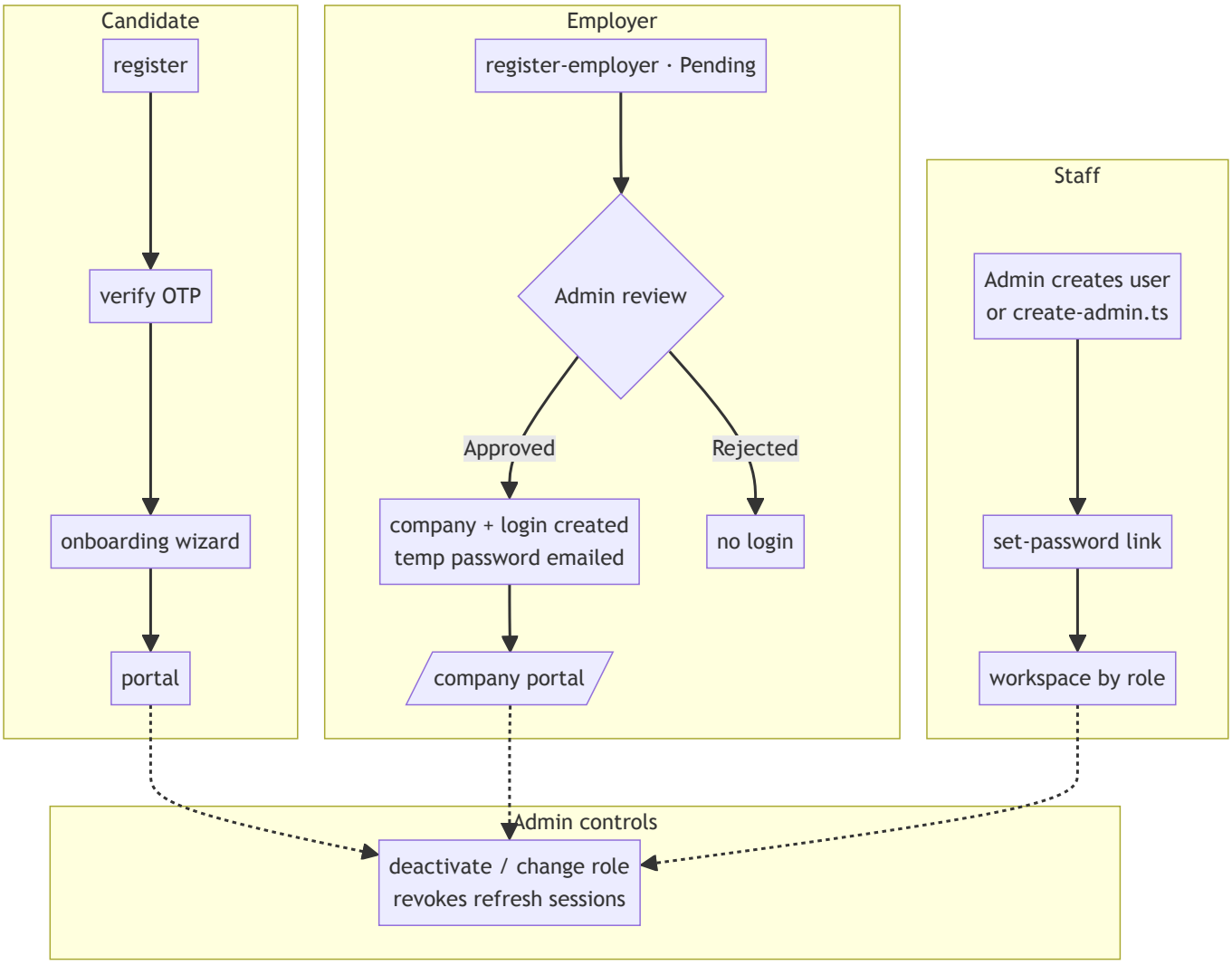

round created
(employer request, Q3 round-setup, next-round decision)
Pending
Q3 requests slots / edits panel
Q3 confirm slot (invites)
or candidate select-slot
Scheduled
Q3 reschedule before start
re-confirm
Q3 reschedule after start, candidate absent
re-confirm
Q3 conducted (result due 24h)
RescheduleRequested
NoShow
Completed
decision — NextRound / FinalRound / Hold / Rejected / Selected

Two writers record outcomes. Q3 uses `decision`, `decidedAt`, `isFinal` and `ApplicationHiring`. The employer and `/recruitment` path (`submitResult`) uses `result` only. That split is the root cause of TESTING_AND_ISSUES W-01.

4. Request lifecycle (any API call)


Browser (React + TanStack Query)
axios instance (lib/axios.ts)
nginx (deploy/nginx.conf)
NestJS (main.ts)
Guards
Service
Postgres / Redis / Storage
useQuery / useMutation
/api/... + Bearer access token (+ X-CSRF-Token on writes)
proxy_pass api:4000 (X-Forwarded-For)
Throttler → JwtAuth (@Public?) → Roles
429 / 401 / 403
ValidationPipe(DTO) → controller → service
Prisma queries / $transaction
result or HttpException
JSON (AllExceptionsFilter on error)
on 401 → POST /auth/refresh once, replay, on failure → /login
data → cache, invalidate feature key on mutations
Browser (React + TanStack Query)
axios instance (lib/axios.ts)
nginx (deploy/nginx.conf)
NestJS (main.ts)
Guards
Service
Postgres / Redis / Storage

5. Account lifecycle per role



Related documents: COMPLETE_WORKFLOW.md (step-by-step), ROLE_PERMISSION_MATRIX.md, API_DOCUMENTATION.md, DATABASE_SCHEMA.md.